

Lab 7: Docker Apache Containerization

Lab 7: Docker Apache Containerization

This article walks you through installing Docker. It will then show you how to set up a containerized version of the Apache web server application. It includes all of the necessary commands and includes the screenshots of the output of the commands and pictures of the web pages.

Author

Joshua Draper

draperj2

VM Snapshot(s)

Virtual Machine 1542 (CIT371-001-Fall2025-draperj2-Ubuntu22Server) on node 'COIVMHOST2'
cit371-001-fall2025
Start

Summary
Console
Hardware
Cloud-Init
Options
Task History
Snapshots
Firewall

Take Snapshot
Rollback
Edit
Remove

Name	RAM	Date/Status	Description
Day1	No	2025-08-07 10:07:19	Used to revert VM to initial state. Do not delete!
└─ Lab2	No	2025-09-04 12:44:29	Lab2
└─┬─ Lab3	No	2025-09-12 17:34:53	Lab3
└─┬─┬─ Lab4	No	2025-09-13 11:00:54	Lab 4
└─┬─┬─┬─ Lab6	No	2025-10-01 13:53:21	Lab 6
└─┬─┬─┬─┬─ Lab7	No	2025-10-08 13:48:03	Lab 7
└─┬─┬─┬─┬─┬─ NOW			You are here!

Overview

Docker's primary purpose is to simplify and standardize the process of building, sharing, and running applications by using containers. These containers package an application and all its dependencies into a single, isolated unit. Some of the key uses of Docker include Application Portability, Faster Development and Deployment, Microservices Architecture, Resource Efficiency, Environment Isolation, and Simplified Dependency Management. Ultimately, Docker enables you to separate your applications from your infrastructure so you can deliver software quickly.

Apache's primary purpose is to act as a widely used, open-source web server software that delivers web content to users' browsers by handling HTTP requests. It serves static and dynamic content to users. Some of its key uses include hosting websites (simple static sites and complex websites), supporting various web development stacks, enabling virtual hosting on a single server, and providing a customizable, reliable platform for dynamic and static delivery through a very modular and flexible design.

Some of the reasons one would want to containerize Apache are because of the benefits it provides for web development and infrastructure management. It does this through improved portability, consistency, and scalability. It can improve efficiency and resource utilization. It also provides security because the container acts as a sandbox.

Procedure

The first step to take is to sign in to the Proxmox cluster and start the Ubuntu Server and Client; then sign in to the Server and open a terminal.

Part 2: Docker Installation

The first step is to install *curl*, run the following commands:

```
sudo apt update
```

```
sudo apt install curl -y
```

Then move to the student directory with this command:

```
cd /home/student
```

Then download and install Docker with these commands:

```
curl -fsSL https://get.docker.com -o install-docker.sh
```

```
sudo sh install-docker.sh
```

Next verify Docker is working, to do so run this command:

```
sudo docker run hello-world
```

Your output should look like the following:

```
student@371serv:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:54e66cc1dd1fcb1c3c58bd8017914dbed8701e2d8c74d9262e26bd9cc1642d31
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Part 3: Apache Containerization

Next, enter the following commands to create a website directory, and switch to the directory:

```
mkdir /home/student/website
```

```
cd /home/student/website
```

```
pwd
```

Next, we want to run the Apache container, use this command:

```
sudo docker run -dit --name my-apache-app -p 8080:80 -v "$PWD":/usr/local/apache2/htdocs/ httpd:2.4
```

Then we want to create a webpage, enter this command:

```
nano home.htm
```

Then, copy and paste the following script into the nano text editor (Change the text in the <p><p> to your name):

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>371Serv Homepage</h1>
```

```
<p style="font-size:16px;"><i>Joshua Draper</i></p>
```

```
</body>
```

</html>

Next save the file, press Ctrl+O, Enter, Ctrl+X.

Then we need to find the server's IP address, run this command:

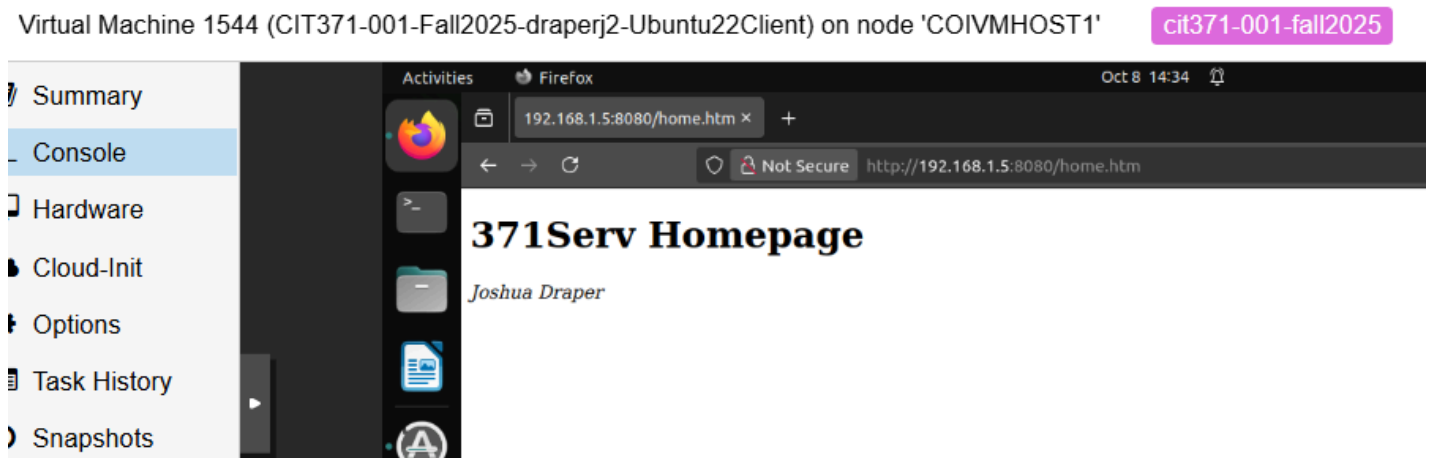
```
ip -br a
```

The correct address should be the one associated with 'ens18'.

Then, switch to your Ubuntu Client and enter the following URL in Firefox (replace 'server_ip' with your server's IP address):

```
http://server_ip:8080/home.htm
```

You should see a page just like this:



Part 4: Apache Container Configuration

Switch back to your Ubuntu Server and enter this command to ensure you are still in /home/student/website:

```
pwd
```

Then we will run these commands to copy the configuration file from the Apache container:

```
sudo docker run --rm httpd:2.4 cat /usr/local/apache2/conf/httpd.conf > my-httpd.conf
```

```
ls
```

Then we want to open the configuration file:

```
nano my-httpd.conf
```

Next, scroll down until you see a list of lines starting with 'Load Module'. Near the bottom of this list find the line `#LoadModule info_module modules/mod_info.so`.

Delete the comment (#) in front of the line to activate it. It should look like this:

```
#LoadModule asis_module modules/mod_asis.so
LoadModule info_module modules/mod_info.so
#LoadModule ssl_module modules/mod_ssl.so
```

Then, go to the very bottom of the file and copy and paste these lines (replace the IP address with your server's IP address):

```
<Location "/server-status">
```

```
    SetHandler server-status
```

```
    Require ip 192.168.1.5
```

```
</Location>
```

```
<Location "/server-info">
```

```
    SetHandler server-info
```

```
    Require ip 192.168.1.5
```

```
</Location>
```

It should look like this:

```
<Location "/server-status">
    SetHandler server-status
    Require ip 192.168.1.5
</Location>
<Location "/server-info">
    SetHandler server-info
    Require ip 192.168.1.5
</Location>
```

Then save the file by pressing Ctrl+O, Enter, Ctrl+X.

Then, to find the container ID enter this command:

```
sudo docker ps -a
```

Then, use these commands to add the file to the Apache container and restart it to apply them (replace containerID with your ID):

```
sudo docker cp my-httpd.conf containerID:/usr/local/apache2/conf/httpd.conf
```

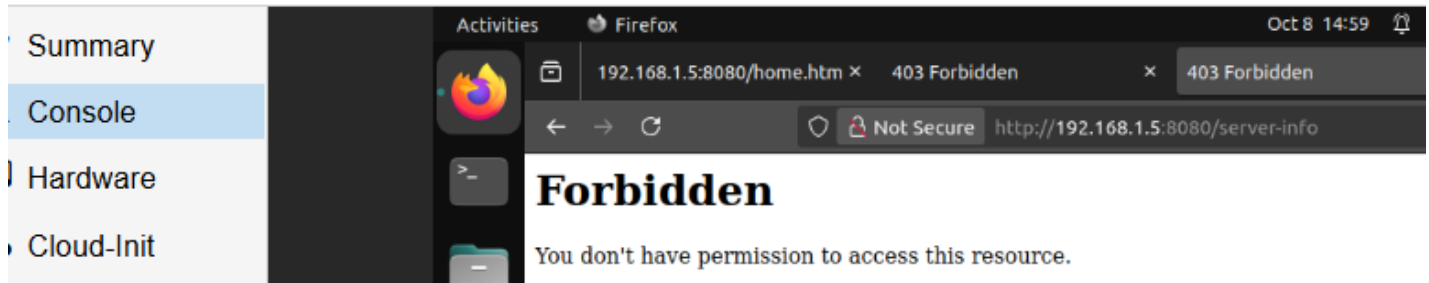
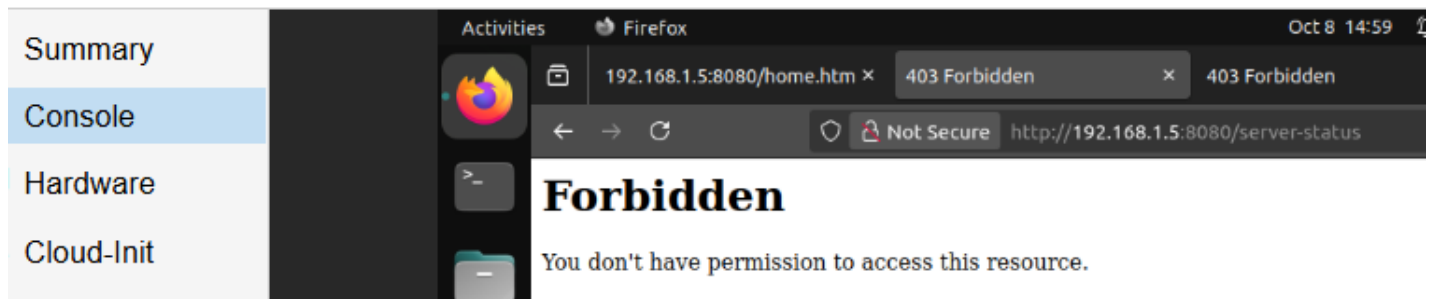
```
sudo docker restart containerID
```

Then, to test it, enter the following URLs on your Server and Client in Firefox (replace server_ip with your IP):

```
http://server_ip:8080/server-status
```

The pages should show up on your Server like this:

The screenshot shows a virtual machine interface. On the left is a sidebar with menu items: Summary, Console (selected), Hardware, Cloud-Init, Options, Task History, Snapshots, and Firewall. Below the menu are icons for VNC, terminal, CD-ROM, and other VM controls. The main window displays a Firefox browser at the URL http://192.168.1.5:8080/server-status. The page title is "Apache Server Status for 192.168.1.5 (via 172.17.0.2)". It lists server details like version (2.4.65) and MPM (event). A status summary indicates the current time, restart time, and various metrics like uptime, load, and CPU usage. A table provides a detailed view of connection slots, threads, and async connections. At the bottom, there's a scoreboard key explaining the symbols used in the table.



Part 5: Docker Container Verification

Finally, we need to run the following command to ensure that only the hello-world and one Apache container are present:

```
sudo docker ps -a
```

Your output should look like this:

```
student@371serv:~/website$ sudo docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        NAMES
bfa7d1037057   httpd:2.4     "httpd-foreground"      39 minutes ago
Up 9 minutes   cp            my-apache-app
764d7b6d5e89   hello-world   "/hello"                 49 minutes ago
Exited (0) 49 minutes ago
nifty_haibt
```

You have now successfully completed the lab! Now just exit the terminal and sign out of the Proxmox cluster.

References

Docker Docs:

[What is Docker? | Docker Docs](#) [↗](#)

GreenGeeks:

[What is Apache and What Does it Do for Website Development?](#) [↗](#)

Sumo Logic:

[What is apache? in-depth overview of apache web server](#) 

Powered by [Wiki.js](#)